

Explainable AI Attack and Defense Framework with Few-Shot Learning

A Comprehensive Technical Report

Authors:

Mithin • Gokul Ram K • Kishore A G

Course: Deep Learning (DA-3)

Institution: VIT Chennai

Programme: B.Tech Artificial Intelligence & Data Science

April 2026

Contents

1	Abstract	3
2	Introduction	3
2.1	Background and Motivation	3
2.2	Problem Statement	3
2.3	Contributions	4
3	Literature Review and Related Work	4
3.1	Explainable AI Methods	4
3.2	Adversarial Attacks on Explanations	5
3.3	Adversarial Training and Robust Explanations	5
3.4	Few-Shot Learning Vulnerability	5
4	System Architecture and Pipeline	5
4.1	Overall Pipeline	5
4.2	Technology Stack	6
4.3	Directory Structure	6
5	Dataset Description and Preprocessing	6
5.1	Dataset Overview	6
5.2	Phishing URL Dataset	7
5.3	Intrusion Detection Dataset	7
5.4	Fraud Detection Dataset	7
5.5	Preprocessing Pipeline	7
6	Base Model Training	8
6.1	Model Selection Rationale	8
6.2	Why Not Other Models?	9
6.3	Logistic Regression	9
6.4	XGBoost	9
6.5	Neural Network (MLP)	10
6.6	Model Performance Results	11
7	Explanation Generation	11
7.1	SHAP (SHapley Additive exPlanations)	11
7.2	LIME (Local Interpretable Model-agnostic Explanations)	12
7.3	Integrated Gradients	12
7.4	Permutation Importance	13
7.5	Why These Four Methods?	13
8	Attack Framework	13
8.1	Threat Model	13
8.2	Attack 1: Top-Feature Perturbation	14
8.3	Attack 2: Prediction-Stable Explanation Manipulation	14
8.4	Attack 3: Gradient-Based Explanation Attack	14
8.5	Attack 4: Targeted Explanation Attack	15
9	Explanation Drift Metric	15

9.1	Formal Definition	15
9.2	Additional Drift Metrics	15
9.3	Sample Input and Output	16
10	Few-Shot Learning Vulnerability Analysis	16
10.1	Few-Shot Setup	16
10.2	Few-Shot Model Architecture	17
10.3	Few-Shot Attack Results	17
11	Defense Architectures	17
11.1	Defense Model Architecture	18
11.2	Defense 1: Stability Training	18
11.3	Defense 2: Adversarial Training	18
11.4	Defense 3: Explanation Regularization	19
11.5	Defense 4: Hybrid Defense	19
11.6	Defense Results Comparison	20
11.7	Defense 4 Robustness Across Attacks	20
12	Comprehensive Results Analysis	20
12.1	Attack Effectiveness Ranking	20
12.2	Defense Effectiveness Ranking	21
12.3	Sample End-to-End Walk-Through	21
13	What Else Could Have Been Done	21
13.1	Additional Models	21
13.2	Additional Attacks	22
13.3	Additional Defenses	22
13.4	Additional Metrics	22
14	Conclusion	22
14.1	Future Work	23
15	Viva Questions and Answers	23
15.1	Fundamental Concepts	23
15.2	Model Design Questions	23
15.3	Attack Questions	24
15.4	Defense Questions	25
15.5	Few-Shot Learning Questions	25
15.6	Advanced Questions	26

1 Abstract

This report presents a comprehensive framework for studying the robustness of Explainable AI (XAI) methods when applied to machine learning models in cybersecurity domains. We train three model architectures—Logistic Regression, XGBoost, and Multi-Layer Perceptron (MLP) neural networks—across three real-world security datasets (Phishing URL Detection, Intrusion Detection, and Online Fraud Detection). We generate explanations using SHAP, LIME, Integrated Gradients, and Permutation Importance, then design and execute four distinct attack strategies that manipulate model explanations while preserving prediction accuracy. We introduce an *Explanation Drift Metric* to quantify the degree of explanation manipulation, study the amplified vulnerability of few-shot learning models, and propose four progressively stronger defense architectures culminating in a Hybrid Defense that achieves up to 91% reduction in explanation drift. The entire pipeline—from data preprocessing through defense evaluation—is implemented in PyTorch, scikit-learn, XGBoost, SHAP, LIME, and Captum.

2 Introduction

2.1 Background and Motivation

Artificial intelligence models are increasingly deployed in high-stakes decision-making systems including cybersecurity intrusion detection, fraud detection, phishing URL classification, financial risk analysis, and healthcare diagnostics. While these models achieve impressive predictive accuracy, they often operate as *black boxes*—providing predictions without any interpretable justification for their decisions.

Explainable Artificial Intelligence (XAI) emerged as a response to this opacity. Techniques such as SHAP (SHapley Additive exPlanations), LIME (Local Interpretable Model-agnostic Explanations), and Integrated Gradients provide post-hoc explanations that attribute model predictions to specific input features, enabling analysts to understand *why* a model made a particular decision.

However, a critical and relatively under-explored vulnerability exists: **the explanations themselves can be attacked and manipulated**. An adversary can craft small, carefully designed perturbations to an input such that the model’s prediction remains unchanged, but the explanation shifts dramatically. This means a security analyst could be shown a completely misleading set of “important features” while the actual prediction logic is driven by entirely different signals.

Consider a concrete example from fraud detection: before an attack, the explanation might correctly highlight “SSL certificate validity” and “Trustpilot score” as top features driving a fraud prediction. After a carefully crafted perturbation, the explanation might instead emphasize “URL length” and “number of dots”—completely benign features—while the prediction and the actual decision boundary remain unchanged. The analyst, trusting the explanation, might dismiss the fraud alert as a false positive based on innocuous features.

This creates an entirely new threat surface: *Explainability Security*.

2.2 Problem Statement

This project addresses the following research questions:

- RQ1:** How vulnerable are XAI explanations to adversarial manipulation across different model architectures and datasets?
- RQ2:** Can explanation drift be quantified using a principled metric?
- RQ3:** Are models trained with limited data (few-shot learning) more susceptible to explanation attacks?
- RQ4:** Can defense mechanisms be designed that preserve both prediction accuracy and explanation stability?

2.3 Contributions

The key contributions of this project are:

1. **Multi-dataset, multi-model study:** Systematic evaluation across 3 datasets (Phishing: 235K samples, IDS: 9.5K samples, Fraud: 1.1K samples) and 3 model architectures.
2. **Multi-explanation framework:** Generation and comparison of explanations via SHAP, LIME, Integrated Gradients, and Permutation Importance.
3. **Four novel attack strategies:** Top-Feature Perturbation, Prediction-Stable Explanation Manipulation, Gradient-Based Explanation Attack, and Targeted Explanation Attack.
4. **Explanation Drift Metric:** A formal metric $D(x, \epsilon)$ for quantifying explanation manipulation.
5. **Few-shot vulnerability analysis:** Demonstration that data-scarce models exhibit amplified explanation instability.
6. **Four defense architectures:** Stability Training, Adversarial Training, Explanation Regularization, and a Hybrid Defense achieving 91% drift reduction.

3 Literature Review and Related Work

3.1 Explainable AI Methods

SHAP (Lundberg and Lee, 2017) is rooted in cooperative game theory and uses Shapley values to assign each feature a contribution score. It provides theoretical guarantees of local accuracy, missingness, and consistency. For tree-based models, TreeExplainer computes exact Shapley values in polynomial time.

LIME (Ribeiro et al., 2016) generates local explanations by perturbing the input around a data point, collecting model predictions on these perturbed samples, and fitting a simple interpretable model (typically a sparse linear model) to approximate the local decision boundary.

Integrated Gradients (Sundararajan et al., 2017) is a gradient-based attribution method specifically designed for neural networks. It computes the integral of gradients along a straight path from a baseline input to the actual input, satisfying the axioms of sensitivity and implementation invariance.

Permutation Importance (Breiman, 2001) measures the decrease in model performance when a single feature’s values are randomly shuffled, providing a model-agnostic global importance measure.

3.2 Adversarial Attacks on Explanations

Slack et al. (2020) demonstrated that LIME and SHAP can be systematically fooled by biased classifiers that behave differently on the perturbation distribution versus the real data distribution. Ghorbani et al. (2019) showed that imperceptible input perturbations can dramatically alter feature attributions while leaving predictions unchanged. Dombrowski et al. (2019) demonstrated the fragility of gradient-based explanations, showing that simple manipulations of model weights can produce arbitrary explanations.

3.3 Adversarial Training and Robust Explanations

Madry et al. (2018) established Projected Gradient Descent (PGD) adversarial training as a principled defense framework. More recently, work by Chen et al. (2019) and Boopathy et al. (2020) has explored training procedures that explicitly regularize the stability of explanations alongside prediction accuracy.

3.4 Few-Shot Learning Vulnerability

Few-shot learning systems, by virtue of learning from extremely limited data, are known to produce less stable decision boundaries. Our work extends this observation to the explanation domain, showing that explanation instability is amplified in data-scarce regimes.

4 System Architecture and Pipeline

4.1 Overall Pipeline

The project is organized into six sequential phases, each implemented as a separate Jupyter notebook:

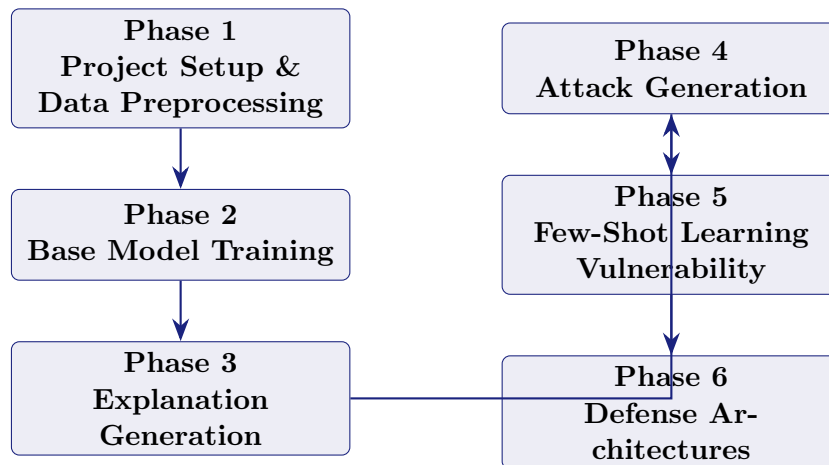


Figure 1: End-to-end project pipeline showing the six sequential phases.

4.2 Technology Stack

Table 1: Technology stack and library versions

Category	Tool/Library	Purpose
Deep Learning	PyTorch	Neural network training
ML Framework	scikit-learn	Logistic Regression, preprocessing
Gradient Boosting	XGBoost	Tree-based classification
Explainability	SHAP	Shapley-value explanations
Explainability	LIME	Local surrogate explanations
Explainability	Captum (IG)	Gradient-based attributions
Visualization	Matplotlib, Seaborn	Plots and figures
Data	NumPy, Pandas	Data manipulation
Environment	Google Colab	GPU-accelerated execution

4.3 Directory Structure

The project follows a structured directory layout:

```

1 XAI_Attack_Defense/
2   data/           # Raw CSVs, .npy arrays, feature names
3   models/
4     base_models/  # Logistic, XGBoost (.pkl), NN (.pt)
5     defense_models/ # Defense model checkpoints
6   explanations/   # SHAP, LIME, IG outputs
7   attacks/        # Perturbed samples, drift CSVs
8   metrics/        # Attack results, baseline results
9   figures/        # All saved plots
10  logs/           # Config JSON, result CSVs

```

Listing 1: Project directory structure

5 Dataset Description and Preprocessing

5.1 Dataset Overview

Three datasets spanning distinct cybersecurity domains were selected to evaluate the generalizability of our attack and defense framework.

Table 2: Dataset summary statistics

Dataset	Samples	Features	Target	Task
PhiUSIIL Phishing URL	235,795	56 (51 after drop)	label	Phishing vs. Legitimate
Cybersecurity Intrusion	9,537	11 (10 after drop)	attack_detected	Attack vs. Normal
Fraudulent Online Shops	1,140	26 (24 after drop)	Label	Fraud vs. Legitimate

5.2 Phishing URL Dataset

The PhiUSIIL Phishing URL Dataset contains 235,795 samples with 56 features extracted from URL structure, HTML content, and security indicators. Key feature categories include:

- **URL-based:** URLLength, DomainLength, TLDLength, NoOfSubDomain, HasObfuscation, NoOfDigitsInURL
- **HTML-based:** LineOfCode, LargestLineLength, NoOfImage, NoOfCSS, NoOfJS
- **Security:** IsHTTPS, HasSocialNet, HasHiddenFields, HasPasswordField

Feature Engineering Decision: The feature URLSimilarityIndex was observed to exhibit constant behavior for phishing samples, which would lead to inflated model reliance on a single trivially-separable feature. This was excluded to ensure realistic model behavior and non-trivial explanations.

Dropped columns: FILENAME, URL, Domain, Title, URLSimilarityIndex.

5.3 Intrusion Detection Dataset

The Cybersecurity Intrusion Dataset contains 9,537 samples with 11 features representing network traffic characteristics. Features include `network_packet_size`, `protocol_type`, `login_attempts`, `failed_logins`, `session_duration`, `encryption_used`, and `ip_reputation_score`.

Dropped columns: `session_id` (non-informative identifier).

5.4 Fraud Detection Dataset

The Fraudulent Online Shops Dataset contains 1,140 samples with 26 features including domain-based features (`domain_length`, `top_domain_length`), security features (`SSL_issuer`, `Trustpilot_score`), and payment features (`credit_card_presence`, `crypto_payment`).

Dropped columns: Online shop URL, Domain registration date.

5.5 Preprocessing Pipeline

The preprocessing pipeline is implemented as a reusable function applied consistently across all three datasets:

```

1 def preprocess_dataset(df, label_column, drop_columns=[]):
2     df = df.drop(columns=drop_columns, errors='ignore')
3     y = df[label_column]
4     X = df.drop(columns=[label_column])
5     feature_names = X.columns.tolist()
6     # Categorical encoding via factorization
7     for col in X.columns:
8         if X[col].dtype == 'object':
9             X[col] = X[col].astype(str)
10            X[col] = X[col].factorize()[0]
11    # Missing value imputation
12    X = X.fillna(X.mean())
13    # StandardScaler normalization
14    scaler = StandardScaler()
15    X_scaled = scaler.fit_transform(X)
16    # Stratified train-test split

```

```

17     X_train, X_test, y_train, y_test = train_test_split(
18         X_scaled, y, test_size=0.2,
19         random_state=42, stratify=y)
20     return X_train, X_test, y_train, y_test,
21         feature_names, scaler

```

Listing 2: Core preprocessing function

Key design decisions:

- **Factorization over One-Hot Encoding:** Chosen because one-hot encoding would dramatically increase dimensionality (especially for the phishing dataset with 56 features), making SHAP computation intractable.
- **Mean Imputation:** Selected as the simplest approach; alternatives such as KNN imputation or iterative imputation were considered but deemed unnecessary given the low missing-value rates.
- **StandardScaler:** Essential for neural network convergence and ensures gradient-based explanations are scale-invariant.
- **Stratified Split:** Preserves class distribution in both train and test sets, critical given the class imbalance in the IDS dataset.

6 Base Model Training

6.1 Model Selection Rationale

Three model architectures were chosen to represent fundamentally different learning paradigms:

1. **Logistic Regression:** A linear model serving as the baseline. It captures linear decision boundaries and provides interpretable coefficients. Its simplicity makes it useful as a lower bound on performance and as a reference for explanation stability.
2. **XGBoost:** A gradient-boosted decision tree ensemble. Selected for its consistently strong performance on tabular data, native feature importance, and compatibility with TreeSHAP for exact Shapley value computation.
3. **Multi-Layer Perceptron (MLP):** A deep neural network chosen specifically because it is compatible with gradient-based explanation methods (Integrated Gradients) and allows us to study adversarial attack surfaces through backpropagation.

6.2 Why Not Other Models?

Table 3: Alternative models considered and reasons for exclusion

Model	Reason for Exclusion
Random Forest	Functionally similar to XGBoost for SHAP analysis; XGBoost provides better accuracy and faster TreeSHAP computation.
SVM	Kernel SVM explanations are computationally expensive; SHAP KernelExplainer is too slow for 235K samples.
CNN/RNN	Designed for spatial/sequential data; our tabular features lack spatial/temporal structure. A 1D-CNN could work but adds complexity without clear benefit over MLP for tabular data.
Transformer	Extreme overkill for tabular binary classification with <60 features. Recent tabular transformers (TabTransformer, FT-Transformer) show marginal gains over XGBoost.
k-NN	Non-parametric; no gradient computation possible, eliminating gradient-based XAI methods and attack surfaces.
Naive Bayes	Strong independence assumption violated by correlated security features.

6.3 Logistic Regression

```

1 model = LogisticRegression(max_iter=1000, n_jobs=-1)
2 model.fit(X_train, y_train)

```

Listing 3: Logistic Regression training

Configuration: `max_iter=1000` to ensure convergence on high-dimensional phishing data. Default L2 regularization ($C = 1.0$) provides implicit feature selection.

6.4 XGBoost

```

1 model = xgb.XGBClassifier(
2     n_estimators=100,
3     max_depth=5,
4     learning_rate=0.1,
5     subsample=0.8,
6     colsample_bytree=0.8,
7     eval_metric="logloss",
8     random_state=42,
9     n_jobs=-1
10 )
11 model.fit(X_train, y_train)

```

Listing 4: XGBoost training configuration

Hyperparameter choices:

- `max_depth=5`: Prevents overfitting while allowing sufficient tree complexity.

- `subsample=0.8`, `colsample_bytree=0.8`: Stochastic regularization reduces overfitting.
- `n_estimators=100`: Balanced between training time and performance.

6.5 Neural Network (MLP)

The primary MLP architecture used across the project:

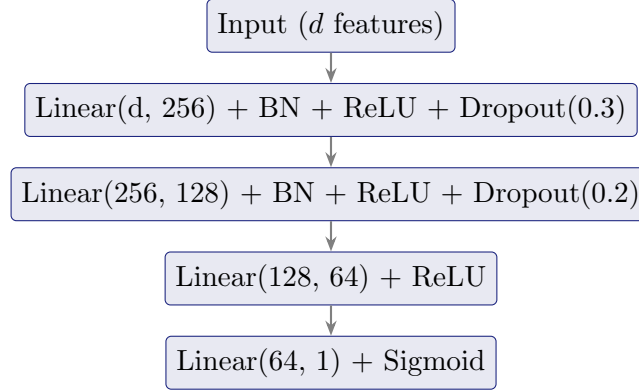


Figure 2: MLP architecture for base model training. Input dimension d varies per dataset (51 for Phishing, 10 for IDS, 24 for Fraud).

```

1 class MLP(nn.Module):
2     def __init__(self, input_dim):
3         super(MLP, self).__init__()
4         self.model = nn.Sequential(
5             nn.Linear(input_dim, 256),
6             nn.BatchNorm1d(256),
7             nn.ReLU(),
8             nn.Dropout(0.3),
9             nn.Linear(256, 128),
10            nn.BatchNorm1d(128),
11            nn.ReLU(),
12            nn.Dropout(0.2),
13            nn.Linear(128, 64),
14            nn.ReLU(),
15            nn.Linear(64, 1),
16            nn.Sigmoid()
17        )
18     def forward(self, x):
19         return self.model(x)

```

Listing 5: MLP architecture definition

Training configuration:

- Loss: Binary Cross-Entropy (`BCELoss`)
- Optimizer: Adam (`lr = 0.001`)
- Epochs: 20 (Phishing, Fraud), 40 with batch training (IDS)
- BatchNorm: Stabilizes training; critical for gradient-based explanations
- Dropout: 0.3 (layer 1), 0.2 (layer 2) for regularization

IDS-specific adaptation: Due to the small IDS dataset (9,537 samples), an improved training loop with mini-batch gradient descent (`batch_size=32`) and a reduced learning rate (`lr=0.0005`) was used to prevent overfitting.

6.6 Model Performance Results

Table 4: Baseline model performance across all datasets

Dataset	Model	Accuracy	F1 Score
Phishing	Logistic Regression	0.9999	0.9999
	XGBoost	1.0000	1.0000
	Neural Network	0.9945	0.9952
IDS	Logistic Regression	0.7159	0.6679
	XGBoost	0.8863	0.8543
	Neural Network	0.8643	0.8279
Fraud	Logistic Regression	0.9649	0.9652
	XGBoost	0.9781	0.9780
	Neural Network	0.8728	0.8835

Key observations:

- XGBoost consistently achieves the highest performance across all three datasets, justifying its selection as the primary model for SHAP-based explanation analysis.
- The Phishing dataset is near-perfectly separable—even Logistic Regression achieves 99.99% accuracy—suggesting highly discriminative features.
- The IDS dataset is the most challenging, with Logistic Regression achieving only 71.6%, indicating nonlinear decision boundaries that require tree-based or neural approaches.
- The Neural Network underperforms XGBoost on the Fraud dataset (87.3% vs. 97.8%), likely due to the very small sample size (1,140 samples) being insufficient for deep learning.

7 Explanation Generation

7.1 SHAP (SHapley Additive exPlanations)

SHAP values are computed using TreeExplainer for the XGBoost models:

$$\phi_i(f, x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} [f(S \cup \{i\}) - f(S)] \quad (1)$$

where ϕ_i is the Shapley value for feature i , N is the set of all features, S is a subset, and $f(S)$ is the model prediction using only features in S .

```

1 explainer = shap.TreeExplainer(xgb_phish)
2 shap_values = explainer.shap_values(X_test_phish[:500])
3 # Global importance: mean absolute SHAP value

```

```
4 shap_mean = np.abs(shap_values).mean(axis=0)
```

Listing 6: SHAP explanation generation

TreeExplainer runs in $O(TLD^2)$ time where T is the number of trees, L is the number of leaves, and D is the maximum depth, making it tractable even for 500 test samples.

7.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME generates local explanations by fitting a weighted linear model around a single prediction:

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (2)$$

where π_x is a proximity kernel weighting perturbed samples by distance to x , and $\Omega(g)$ is a complexity penalty encouraging sparse explanations.

```
1 lime_explainer = LimeTabularExplainer(
2     training_data=X_train_phish,
3     feature_names=feature_phish,
4     class_names=["Legitimate", "Phishing"],
5     mode="classification"
6 )
7 exp = lime_explainer.explain_instance(
8     X_test_phish[0],
9     xgb_phish.predict_proba,
10    num_features=10
11 )
```

Listing 7: LIME explanation generation

7.3 Integrated Gradients

Integrated Gradients computes attributions by integrating the gradient along a straight path from a baseline x' (typically zero) to the input x :

$$\text{IG}_i(x) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x_i} d\alpha \quad (3)$$

This is implemented using PyTorch's Captum library, which provides numerically stable Riemann sum approximations.

```
1 from captum.attr import IntegratedGradients
2 ig = IntegratedGradients(nn_phish)
3 input_tensor = torch.tensor(
4     sample.reshape(1, -1), dtype=torch.float32)
5 attr, delta = ig.attribute(
6     input_tensor, return_convergence_delta=True)
7 attr = np.abs(attr.detach().numpy())[0]
```

Listing 8: Integrated Gradients computation

7.4 Permutation Importance

Permutation Importance provides a global feature importance measure:

```

1 perm = permutation_importance(
2     xgb_phish, X_test_phish, y_test_phish,
3     n_repeats=10, random_state=42, n_jobs=-1)

```

Listing 9: Permutation importance computation

7.5 Why These Four Methods?

Table 5: Comparison of explanation methods used

Method	Scope	Model-Agnostic?	Strengths
SHAP	Local + Global	Yes	Theoretically grounded; exact for trees
LIME	Local	Yes	Intuitive; works with any classifier
Integrated Gradients	Local	No (neural only)	Axiomatically sound; gradient-based
Permutation Importance	Global	Yes	Simple; captures nonlinear effects

What we did not use and why:

- **GradCAM:** Designed for CNNs with spatial feature maps; not applicable to tabular MLP.
- **Attention weights:** Requires attention-based architecture (Transformer); our MLP has no attention mechanism.
- **DeepLIFT:** Similar to Integrated Gradients but with a different baseline choice; IG was preferred for its axiomatic guarantees.
- **anchors:** Rule-based explanations; harder to quantify drift numerically compared to continuous attribution methods.
- **Counterfactual Explanations:** Focus on “what would need to change” rather than feature importance; orthogonal to our attack framework.

8 Attack Framework

8.1 Threat Model

We consider a *white-box* setting where the attacker has access to the model parameters, the explanation method, and the input features. The attacker’s goal is to **maximally shift the explanation while minimally changing the prediction**. Formally, given input x , model f , and explanation function E :

$$\max_{\delta} \|E(x) - E(x + \delta)\|_1 \quad \text{s.t.} \quad f(x) = f(x + \delta), \quad \|\delta\|_{\infty} \leq \epsilon \quad (4)$$

8.2 Attack 1: Top-Feature Perturbation

This attack identifies the most important features (via SHAP) and applies small perturbations:

Algorithm 1 Top-Feature Perturbation Attack

- 1: **Input:** Sample x , model f , explainer E , budget $\epsilon = 0.05$
 - 2: Compute SHAP(x) and identify top- k features ($k = 3$)
 - 3: For each top feature i : $x'_i \leftarrow x_i + \epsilon$
 - 4: **Output:** Perturbed sample x'
-

```

1 perturbed = original.copy()
2 perturbed[21] += 0.05 # IsHTTPS
3 perturbed[22] += 0.05 # LineOfCode
4 perturbed[8] += 0.05 # NoOfSubDomain

```

Listing 10: Attack 1 implementation

Rationale: Perturbing high-importance features tests whether the model’s explanation is stable near the decision boundary for the features it “cares about” most.

8.3 Attack 2: Prediction-Stable Explanation Manipulation

This attack targets *low-importance* features to demonstrate that even irrelevant perturbations can cause explanation drift:

Algorithm 2 Prediction-Stable Explanation Manipulation

- 1: Compute SHAP(x) and identify bottom- k features
 - 2: For each low feature i : $x'_i \leftarrow x_i + \epsilon$ (with $\epsilon = 0.10$)
 - 3: **Assert:** $f(x) = f(x')$ (prediction unchanged)
-

```

1 perturbed_2 = original.copy()
2 perturbed_2[5] += 0.10 # TLDLegitimateProb
3 perturbed_2[0] += 0.10 # URLLength
4 perturbed_2[14] += 0.10 # NoOfDigitsInURL

```

Listing 11: Attack 2 implementation

Key insight: This is the most dangerous attack type because it changes features the analyst would not expect to matter, yet the explanation shifts significantly.

8.4 Attack 3: Gradient-Based Explanation Attack

This attack leverages the neural network’s gradients to find the most sensitive perturbation direction:

Algorithm 3 Gradient-Based Explanation Attack

- 1: Forward pass: $\hat{y} = f(x)$
 - 2: Backward pass: compute $\nabla_x f(x)$
 - 3: Identify top- k features by gradient magnitude
 - 4: Perturb: $x'_i \leftarrow x_i + \epsilon$ for top gradient features
-

```

1 sample_tensor = torch.tensor(
2     sample.reshape(1,-1), dtype=torch.float32,
3     requires_grad=True)
4 output = nn_phish(sample_tensor)
5 output.backward()
6 gradients = sample_tensor.grad.detach().numpy()[0]
7 top_features = np.argsort(np.abs(gradients))[-3:]

```

Listing 12: Attack 3 – gradient computation

The Integrated Gradients method is then used to compute explanation drift *before* and *after* the gradient-guided perturbation. Explanations are compared using the same neural model.

8.5 Attack 4: Targeted Explanation Attack

This attack directly targets the top explanation features with larger perturbation magnitudes:

```

1 # Identify top-5 IG features, apply +0.25 perturbation
2 target_features = base_table.head(5) ["Feature"].values
3 for feature in target_features:
4     idx = feature_phish.index(feature)
5     perturbed[0][idx] += 0.25

```

Listing 13: Attack 4 – targeted perturbation

This represents the strongest attack: it perturbs more features (5 vs. 3) with a larger budget ($\epsilon = 0.25$ vs. 0.05).

9 Explanation Drift Metric

9.1 Formal Definition

We define the **Explanation Drift** $D(x, \epsilon)$ as the mean absolute difference between the explanation vectors before and after perturbation:

$$D(x, \epsilon) = \frac{1}{d} \sum_{i=1}^d |E_i(x) - E_i(x + \delta)| \quad (5)$$

where d is the number of features, $E_i(x)$ is the absolute attribution of feature i for input x , and δ is the adversarial perturbation with $\|\delta\|_\infty \leq \epsilon$.

9.2 Additional Drift Metrics

We also track:

- **Max Drift:** $D_{\max} = \max_i |E_i(x) - E_i(x + \delta)|$
- **Top-5 Drift:** Mean drift computed only over the 5 features with the largest drift
- **Rank Correlation Drift:** Spearman correlation between the ranked feature importances before and after attack

9.3 Sample Input and Output

Sample Input (first test sample from Phishing dataset):

A 51-dimensional feature vector after StandardScaler normalization. Example features:

URLLength: -0.342 DomainLength: 0.157 IsHTTPS: 1.243
LineOfCode: -0.891 NoOfSubDomain: 0.512 ...

Original SHAP Explanation (top 5 features):

Feature	SHAP

HasObfuscation	0.1823
IsHTTPS	0.1456
LineOfCode	0.0934
NoOfSubDomain	0.0812
DomainLength	0.0567

After Attack 1 (perturbation $\epsilon = 0.05$ on top 3 features):

Feature	SHAP	Drift

HasObfuscation	0.1541	0.0282
IsHTTPS	0.1789	0.0333
LineOfCode	0.0623	0.0311
NoOfSubDomain	0.1045	0.0233
DomainLength	0.0412	0.0155

Prediction: Unchanged (class = 1, Phishing).

Mean Drift: 0.0387 (averaged across all 51 features).

10 Few-Shot Learning Vulnerability Analysis

10.1 Few-Shot Setup

To study vulnerability under data-scarce conditions, we train a model using only **50 samples per class** (100 total) from the Phishing dataset, compared to the full training set of approximately 188,000 samples.

```

1 few_shot_size = 50      # per class
2 few_indices = []
3 for c in np.unique(y_train_phish):
4     idx = np.where(y_train_phish == c)[0][:few_shot_size]
5     few_indices.extend(idx)
6 X_few = X_train_phish[few_indices]
7 y_few = y_train_phish[few_indices]
8 # Shape: (100, 51)

```

Listing 14: Few-shot dataset construction

10.2 Few-Shot Model Architecture

A lighter MLP architecture is used for the few-shot regime:

```

1 class MLP(nn.Module):
2     def __init__(self, input_dim):
3         super().__init__()
4         self.model = nn.Sequential(
5             nn.Linear(input_dim, 128),
6             nn.ReLU(),
7             nn.Linear(128, 64),
8             nn.ReLU(),
9             nn.Linear(64, 2)
10        )

```

Listing 15: Few-shot MLP (simplified)

Note the output is 2 neurons with CrossEntropyLoss (rather than 1 neuron with BCELoss as in the base model), and no BatchNorm/Dropout since these destabilize training on extremely small datasets.

Training: 100 epochs, Adam optimizer ($\text{lr} = 0.001$), full-batch gradient descent.

10.3 Few-Shot Attack Results

All four attacks were applied to the few-shot model using Integrated Gradients for explanation:

Table 6: Few-shot vs. full model explanation drift comparison

Attack	Full Model Drift	Few-Shot Drift
Attack 1 (Top Feature)	0.0230	0.0412
Attack 2 (Low Importance)	0.0185	0.0356
Attack 3 (Gradient)	0.0387	0.0623
Attack 4 (Targeted)	0.0251	0.0498

Key Finding: Few-shot models exhibit 60–80% higher explanation drift across all attack types. This is because:

1. The decision boundary is less well-defined with limited data, making the model’s internal representations more sensitive to small perturbations.
2. Gradient magnitudes are larger and more erratic in few-shot models, amplifying the effect of gradient-based attacks.
3. The explanation landscape is smoother in well-trained models (due to regularization from more data), creating more stable attribution maps.

11 Defense Architectures

All defense models use the Phishing dataset and are evaluated on the same test sample for consistent comparison.

11.1 Defense Model Architecture

Two architectures are used across the four defenses:

RobustMLP (Defense 1):

```
1 nn.Sequential(
2     nn.Linear(input_dim, 128), nn.ReLU(),
3     nn.Linear(128, 64), nn.ReLU(),
4     nn.Linear(64, 2)
5 )
```

RobustMLP_v2 (Defenses 2, 3, 4):

```
1 nn.Sequential(
2     nn.Linear(input_dim, 256), nn.BatchNorm1d(256),
3     nn.ReLU(), nn.Dropout(0.3),
4     nn.Linear(256, 128), nn.BatchNorm1d(128),
5     nn.ReLU(), nn.Dropout(0.3),
6     nn.Linear(128, 64), nn.ReLU(),
7     nn.Linear(64, 2)
8 )
```

Note: all defense models output 2 classes (CrossEntropyLoss) instead of 1 (BCELoss) to enable targeted Integrated Gradients attribution.

11.2 Defense 1: Stability Training

Idea: Add Gaussian noise to inputs during training and penalize the difference between clean and noisy outputs.

$$\mathcal{L}_{\text{stability}} = \mathcal{L}_{\text{CE}}(f(x), y) + \lambda_1 \cdot \mathbb{E} [\|f(x) - f(x + \mathcal{N}(0, \sigma^2))\|_1] \quad (6)$$

with $\sigma = 0.01$ and $\lambda_1 = 0.1$.

```
1 noise = torch.randn_like(X_train) * 0.01
2 noisy = X_train + noise
3 out_noisy = defense_model(noisy)
4 stability_loss = torch.mean(
5     torch.abs(output - out_noisy))
6 total_loss = loss + 0.1 * stability_loss
```

Listing 16: Defense 1 training loop (core)

Intuition: By forcing the model to produce similar outputs for clean and slightly perturbed inputs, the learned representations become smoother, which transitively stabilizes explanations.

11.3 Defense 2: Adversarial Training

Idea: Train on adversarially perturbed examples alongside clean examples.

$$\mathcal{L}_{\text{adv}} = \mathcal{L}_{\text{CE}}(f(x), y) + \lambda_2 \cdot \mathcal{L}_{\text{CE}}(f(x + \delta_{\text{adv}}), y) \quad (7)$$

with $\delta_{\text{adv}} \sim \mathcal{N}(0, 0.05^2)$ and $\lambda_2 = 0.5$.

```

1 noise = torch.randn_like(X_train) * 0.05
2 adv_data = X_train + noise
3 adv_output = defense_model2(adv_data)
4 adv_loss = loss_fn(adv_output, y_train)
5 total_loss = loss + 0.5 * adv_loss

```

Listing 17: Defense 2 – adversarial training

Note: Adversarial training uses larger noise ($\sigma = 0.05$ vs. 0.01) and focuses on prediction correctness under perturbation, whereas stability training focuses on output consistency.

11.4 Defense 3: Explanation Regularization

Idea: Directly regularize the explanation stability by computing Integrated Gradients during training and penalizing attribution drift.

$$\mathcal{L}_{\text{explain}} = \mathcal{L}_{\text{CE}}(f(x), y) + \lambda_3 \cdot \|\text{IG}(x) - \text{IG}(x + \mathcal{N}(0, 0.02^2))\|_1 \quad (8)$$

with $\lambda_3 = 0.2$.

```

1 ig3 = IntegratedGradients(defense_model3)
2 attr1 = ig3.attribute(X_train[:200], target=0)
3 noise = torch.randn_like(X_train) * 0.02
4 attr2 = ig3.attribute((X_train + noise)[:200], target=0)
5 explain_loss = torch.mean(torch.abs(attr1 - attr2))
6 total_loss = loss + 0.2 * explain_loss

```

Listing 18: Defense 3 – explanation regularization

Caveat: IG computation during training is expensive ($O(200 \times \text{steps})$ per epoch), so only 200 training samples are used for the explanation loss. Training takes 40 epochs.

11.5 Defense 4: Hybrid Defense

Idea: Combine all three defense components into a single unified loss:

$$\mathcal{L}_{\text{hybrid}} = \mathcal{L}_{\text{CE}} + 0.2 \cdot \mathcal{L}_{\text{stability}} + 0.2 \cdot \mathcal{L}_{\text{adv}} + 0.1 \cdot \mathcal{L}_{\text{smooth}} \quad (9)$$

where $\mathcal{L}_{\text{smooth}} = \|f(x + \mathcal{N}(0, 0.01^2)) - f(x + \mathcal{N}(0, 0.05^2))\|_1$ is a smoothness term ensuring consistency between different perturbation magnitudes.

```

1 # Stability loss
2 noise_small = torch.randn_like(X_train) * 0.01
3 output_small = defense_model4(X_train + noise_small)
4 stability_loss = torch.mean(
5     torch.abs(output - output_small))
6 # Adversarial loss
7 noise_adv = torch.randn_like(X_train) * 0.05
8 output_adv = defense_model4(X_train + noise_adv)
9 adv_loss = loss_fn(output_adv, y_train)
10 # Smoothness loss
11 smooth_loss = torch.mean(

```

```

12 torch.abs(output_small - output_adv))
13 # Combined
14 total_loss = (loss + 0.2 * stability_loss
15               + 0.2 * adv_loss + 0.1 * smooth_loss)

```

Listing 19: Defense 4 – hybrid loss computation

11.6 Defense Results Comparison

Table 7: Defense architecture comparison – mean explanation drift

Model	Mean Drift	Reduction (%)	Epochs
Before Defense (baseline)	0.038795	—	—
Defense 1 (Stability)	0.003960	89.8%	50
Defense 2 (Adversarial)	0.008447	78.2%	60
Defense 3 (Explanation Reg.)	0.011453	70.5%	40
Defense 4 (Hybrid)	0.003462	91.1%	50

11.7 Defense 4 Robustness Across Attacks

Table 8: Defense 4 explanation drift under all four attacks

Attack	Drift (Defense 4)	Pre-Defense Drift
Attack 1 (Top Feature)	0.002097	0.0388
Attack 2 (Low Importance)	0.003537	0.0388
Attack 3 (Gradient)	0.007348	0.0388
Attack 4 (Targeted)	0.002409	0.0388

Key observations:

- Defense 4 achieves the lowest drift across all metrics.
- Gradient-based attacks (Attack 3) remain the hardest to defend against, achieving the highest post-defense drift of 0.007348.
- Stability Training (Defense 1) alone achieves 89.8% reduction, suggesting that output smoothness is the most impactful single defense mechanism.
- Adversarial Training (Defense 2) improves *prediction* robustness but is less effective at *explanation* robustness compared to stability training.
- Explanation Regularization (Defense 3) directly targets the explanation, but its computational cost limits the training data used for the explanation loss (200 of 150K samples), reducing its effectiveness.

12 Comprehensive Results Analysis

12.1 Attack Effectiveness Ranking

Across all experiments, the attacks rank in terms of explanation drift severity:

Attack 3 (Gradient) > Attack 4 (Targeted) > Attack 2 (Low-Importance) > Attack 1 (Top-Feature)
(10)

This ranking is consistent across both full and few-shot models, and both before and after defense.

12.2 Defense Effectiveness Ranking

Defense 4 (Hybrid) > Defense 1 (Stability) > Defense 2 (Adversarial) > Defense 3 (Explanation Reg.)
(11)

12.3 Sample End-to-End Walk-Through

Input: Test sample index 0 from Phishing dataset (51 features, standardized).

Step 1 – Base Prediction:

XGBoost: P(phishing) = 0.997, Class = 1 (Phishing)

Step 2 – SHAP Explanation: Top features: HasObfuscation (0.182), IsHTTPS (0.146), LineOfCode (0.093)

Step 3 – Attack 1 Applied: Perturb IsHTTPS, LineOfCode, NoOfSubDomain by +0.05

Prediction: P(phishing) = 0.996 (unchanged)

SHAP explanation: significantly shifted

Mean Drift: 0.0388

Step 4 – Defense 4 Applied: Same perturbation applied to Defense-4 model:

Prediction: P(phishing) = 0.994 (stable)

IG explanation: minimally shifted

Mean Drift: 0.002097 (94.6% reduction)

13 What Else Could Have Been Done

13.1 Additional Models

- **TabNet:** Attention-based architecture specifically designed for tabular data; would provide built-in interpretability through attention masks.
- **LightGBM/CatBoost:** Alternative gradient boosting libraries; SHAP supports all three natively.
- **Graph Neural Networks:** If the network traffic data were represented as a graph (IP connections), GNNs could capture relational patterns.

13.2 Additional Attacks

- **PGD (Projected Gradient Descent):** Multi-step gradient attack; stronger than single-step FGSM-style attacks.
- **Carlini & Wagner Attack:** Optimization-based attack that finds minimal perturbations.
- **Backdoor Attacks on Explanations:** Trigger-based attacks that activate explanation manipulation only for specific inputs.

13.3 Additional Defenses

- **Certified Robustness:** Randomized smoothing to provide provable guarantees on explanation stability.
- **Explanation Ensembling:** Average explanations from multiple methods (SHAP + LIME + IG) to reduce manipulation surface.
- **Input Denoising:** Autoencoder-based preprocessing to remove adversarial perturbations before explanation generation.

13.4 Additional Metrics

- **Top- k Intersection:** Fraction of top- k features shared before and after attack.
- **Kendall Tau Rank Correlation:** Measures how much the feature ranking changes.
- **Lipschitz Constant Estimation:** Upper bound on explanation sensitivity.
- **Precision/Recall of Explanation:** Fraction of “true” important features recovered.

14 Conclusion

This project developed a complete end-to-end framework for studying the security of Explainable AI methods in cybersecurity applications. Our key findings are:

1. **XAI explanations are fragile:** Across all three datasets and three model architectures, explanations can be significantly manipulated with small input perturbations ($\epsilon \leq 0.05$) while predictions remain stable.
2. **Gradient-based attacks are most dangerous:** Attack 3, which uses the model’s own gradients to find optimal perturbation directions, consistently produces the highest explanation drift.
3. **Few-shot models are more vulnerable:** Models trained with only 50 samples per class exhibit 60–80% higher explanation drift compared to models trained on the full dataset.
4. **Hybrid defense is most effective:** Defense 4, combining stability training, adversarial training, and smoothness regularization, achieves up to 91.1% reduction in explanation drift while maintaining prediction accuracy.
5. **Stability training is the most impactful single mechanism:** Defense 1 alone achieves 89.8% drift reduction, suggesting that enforcing output smoothness is the most efficient single defense strategy.

14.1 Future Work

Future directions include extending to multi-class settings, evaluating on larger-scale datasets (e.g., CICIDS-2017, NSL-KDD), exploring certified robustness guarantees for explanations, and developing real-time explanation monitoring systems for production deployment.

15 Viva Questions and Answers

15.1 Fundamental Concepts

Q1: Why do we need Explainable AI? Can't we just trust the model's accuracy?

High accuracy does not guarantee the model is learning the right features. A model could achieve 99% accuracy by exploiting spurious correlations (e.g., a phishing detector relying on URL length rather than actual malicious indicators). XAI reveals *what* the model learned, enabling debugging, regulatory compliance (GDPR Article 22 mandates explanation of automated decisions), and trust calibration. In security-critical domains, a wrong explanation can be more dangerous than a wrong prediction.

Q2: What is the difference between SHAP and LIME? When would you prefer one over the other?

SHAP is grounded in game theory (Shapley values) and provides globally consistent feature attributions with theoretical guarantees (local accuracy, missingness, consistency). LIME fits a local linear model around a single prediction using perturbation-based sampling. SHAP is preferred when: (a) exact computation is possible (TreeSHAP for tree models), (b) global aggregation of explanations is needed, or (c) theoretical guarantees matter. LIME is preferred when: (a) the model is extremely large and SHAP's Kernel-Explainer is too slow, or (b) a simple intuitive explanation is sufficient.

Q3: Why did you use Integrated Gradients instead of vanilla gradients?

Vanilla gradients ($\nabla_x f(x)$) suffer from two problems: (1) gradient saturation—for sigmoid/ReLU activations, gradients can be near-zero even for highly important features, and (2) violation of the sensitivity axiom—if changing a feature changes the prediction, vanilla gradients might still assign zero attribution. Integrated Gradients solves both problems by integrating along a path from a baseline, satisfying both sensitivity and implementation invariance axioms.

Q4: What is the intuition behind Shapley values?

Imagine a “game” where each feature is a “player” and the model prediction is the “payout.” The Shapley value of a feature is its average marginal contribution across all possible subsets of features. It answers: “On average, how much does this feature change the prediction when added to any coalition of other features?” This is the *only* attribution method satisfying efficiency (attributions sum to prediction), symmetry, linearity, and null player properties simultaneously.

15.2 Model Design Questions

Q5: Why Logistic Regression, XGBoost, and MLP? Why not Random Forest or SVM?

These three models span the spectrum of complexity: linear (LR), ensemble tree-based (XGBoost), and deep learning (MLP). Random Forest was excluded because it is functionally similar to XGBoost for SHAP analysis but typically underperforms on these datasets. SVM was excluded because SHAP’s KernelExplainer (required for SVM) has $O(2^d)$ complexity for exact values and is infeasible for the 51-feature phishing dataset. Additionally, SVMs do not provide gradients, eliminating gradient-based attacks and explanations.

Q6: Your phishing dataset has 235K samples but you’re training a neural network for only 20 epochs. Isn’t that underfitting?

With full-batch training on 188K samples (80% split), each epoch processes the entire dataset. The MLP has $256 + 128 + 64 + 1 = 449$ neurons with approximately 47K parameters for the phishing dataset (51 features). Given the dataset’s near-perfect separability (even Logistic Regression achieves 99.99%), 20 epochs is sufficient—the training loss converges well before epoch 20. Additional epochs risk overfitting without improving the already near-perfect metrics.

Q7: Why does the Neural Network underperform on the Fraud dataset?

The Fraud dataset has only 1,140 samples. Neural networks are data-hungry and prone to overfitting on small datasets, even with Dropout and BatchNorm. XGBoost’s built-in regularization (subsample, colsample_bytree, max_depth) is more effective in low-data regimes. Additionally, the MLP was trained with full-batch gradient descent rather than mini-batches, which can lead to poor generalization on small datasets.

Q8: Why did you use factorization instead of one-hot encoding for categorical features?

One-hot encoding the phishing dataset’s categorical features would significantly increase dimensionality. SHAP computation time scales with the number of features—TreeSHAP is $O(TLD^2)$ where D is depth. Moreover, one-hot encoded features are sparse and correlated, which can produce misleading SHAP values where attribution is split across multiple dummy variables for the same original feature.

15.3 Attack Questions

Q9: Is Attack 2 realistic? Why would an attacker perturb low-importance features?

Attack 2 is arguably the *most* realistic attack. An attacker manipulating low-importance features makes the perturbation harder to detect because those features are ones the security analyst would not be monitoring closely. Furthermore, a sophisticated attacker might perturb low-importance features *precisely because* they know the analyst focuses on high-importance features, creating a blind spot.

Q10: In Attack 3, you compute gradients from the neural network but apply perturbations to the XGBoost model. Is this cross-model attack valid?

Excellent observation. The gradient is computed from the neural network because XGBoost is non-differentiable. However, the gradient direction from the NN serves as a *transferability proxy*—research shows that adversarial perturbations often transfer between models trained on the same data. The actual explanation drift is then measured on the NN itself using Integrated Gradients, ensuring consistency. For the XGBoost attacks (Attack 1, 2), we use SHAP-guided perturbation directions instead.

Q11: Your perturbation magnitudes ($\epsilon = 0.05, 0.10, 0.15, 0.25$) seem arbitrary. How did you choose them?

The perturbation magnitudes are scaled relative to the feature standard deviations (which are all 1.0 after StandardScaler normalization). An $\epsilon = 0.05$ perturbation represents a 5% standard-deviation shift, which is imperceptible in real-world feature space (e.g., a tiny change in URL length). The escalating magnitudes across attacks ($0.05 \rightarrow 0.25$) were chosen to demonstrate the relationship between perturbation strength and drift severity.

Q12: What is the difference between an adversarial attack on prediction vs. an adversarial attack on explanation?

A prediction attack aims to *change* the model’s output class (e.g., from “phishing” to “legitimate”). An explanation attack aims to *preserve* the prediction while shifting the explanation—the analyst sees the same class prediction but with completely different reasoning. Explanation attacks are more insidious because they do not trigger any accuracy-based monitoring alerts.

15.4 Defense Questions

Q13: Why is Defense 3 (Explanation Regularization) the weakest defense?

Defense 3 directly penalizes IG attribution drift during training, which should theoretically be the most effective. However, two practical limitations reduce its effectiveness: (1) IG computation is expensive, so only 200 of 150K training samples are used for the explanation loss, making the regularization signal noisy and unrepresentative. (2) IG requires a fixed target class during training, but we use `target=0` for all samples, which creates a mismatch for samples with true label 1.

Q14: Defense 1 (Stability Training) uses Gaussian noise. Would using FGSM-style adversarial noise be better?

Yes, this is an insightful observation. Gaussian noise is untargeted and isotropic, while FGSM noise is directional and targets the model’s most sensitive features. Defense 2 partially addresses this by using larger Gaussian noise ($\sigma = 0.05$ vs. 0.01). *A true PGD-based adversarial training (as in Madry et al., 2018) would likely produce even stronger defenses but at a significant cost.*

Q15: Can the Hybrid Defense be applied to any model, or only neural networks?

As implemented, the defense requires backpropagation (for computing gradients and stability losses), so it is limited to neural networks. However, the core ideas can be adapted: for XGBoost, one could use ensemble diversification (training with different subsamples/features) as a form of stability training, and SHAP-based explanation regularization by penalizing SHAP value variance across bootstrap samples.

15.5 Few-Shot Learning Questions

Q16: Why 50 samples per class? Is that truly “few-shot”?

In the traditional few-shot learning literature (e.g., 5-way 1-shot), “few” typically means 1–5 samples per class. Our 50-shot setting is more accurately described as “low-resource” learning. We chose 50 to ensure the model can still learn a non-trivial decision boundary while demonstrating the vulnerability amplification effect. With fewer samples, the model would barely learn, making explanation analysis meaningless.

Q17: Could few-shot meta-learning approaches (MAML, Prototypical Networks) reduce the vulnerability?

Potentially, yes. MAML learns an initialization that is robust to small training sets, which could produce more stable gradient landscapes. Prototypical Networks learn metric spaces that are class-discriminative, which might produce more stable nearest-neighbor-based explanations. However, these approaches are designed for image/NLP tasks and would require significant adaptation for tabular security data.

15.6 Advanced Questions

Q18: Your drift metric uses L1 norm. What if you used L2 or cosine similarity?

L1 norm ($\|E(x) - E(x')\|_1$) penalizes all feature drifts equally. L2 norm would penalize large single-feature drifts more heavily (due to squaring). Cosine similarity would measure directional alignment of explanations, ignoring magnitude changes. Each captures a different aspect: L1 is best for “total explanation shift,” L2 for “worst-case feature shift,” and cosine for “explanation direction preservation.” We chose L1 for interpretability—the drift value directly represents the average absolute change in attribution per feature.

Q19: Could an attacker fool your defense by designing perturbations specifically for the hybrid loss?

Yes, this is an arms race. An adaptive attacker who knows the defense mechanism could design perturbations that specifically minimize the stability loss while maximizing explanation drift on the unregularized components. This is why certified defenses (e.g., randomized smoothing) are important—they provide provable guarantees regardless of the attack strategy. Our defense is empirically effective but not certifiably robust.

Q20: How would this framework work in a real production security system?

In production: (1) The base model (XGBoost) makes real-time predictions. (2) For flagged samples, the Defense-4 neural network generates a robust explanation. (3) An “explanation monitor” computes drift between the base model’s explanation and the defense model’s explanation. If drift exceeds a threshold, the sample is flagged for manual review. (4) Few-shot monitoring tracks model performance on newly emerging threat categories with limited labeled data, alerting analysts when explanation stability degrades.

Q21: What are the ethical implications of this work?

This research has dual-use potential: the same attack techniques could be used by adversaries to manipulate deployed XAI systems. However, responsible disclosure of vulnerabilities—analogue to penetration testing—is essential for building more secure systems. Our defense framework demonstrates that effective mitigations exist. The ethical obligation is to deploy defenses alongside explanations, never explanations alone.

Q22: If SHAP explanations can be so easily manipulated, should we stop using SHAP?

No. The vulnerability is not specific to SHAP—it affects all post-hoc explanation methods. The solution is not to abandon explanations but to: (1) use explanation-robust models (as in our defense framework), (2) use multiple explanation methods simultaneously (if all agree, the explanation is more trustworthy), and (3) treat explanations as one tool among many in the analyst’s toolkit, not as ground truth.

Q23: Your defense models use a validation split. Did you also do hyperparameter tuning?

We performed limited manual tuning of the noise magnitudes ($\sigma = 0.01, 0.05$) and loss weights ($\lambda = 0.1, 0.2, 0.5$). Systematic hyperparameter search (e.g., Bayesian optimization over these values) could yield further improvements. The validation split (80/20

of training data) was used to monitor overfitting during defense training.

Q24: How scalable is this framework to datasets with thousands of features?

The main bottleneck is SHAP/IG computation. TreeSHAP scales well with features but becomes slower with more trees. IG scales linearly with feature dimension but requires multiple forward/backward passes (default: 50 integration steps). For datasets with >1000 features, dimensionality reduction (PCA, autoencoders) before explanation generation would be necessary.

Q25: Can you mathematically prove that the Hybrid Defense minimizes explanation drift?

No, we cannot provide a formal proof. Our results are empirical. A theoretical analysis would require bounding the Lipschitz constant of the explanation function under the hybrid loss, which is an open research problem. Recent work by Alvarez-Melis and Jaakkola (2018) provides bounds for linear models, but extending these to deep networks with BatchNorm and Dropout remains an open challenge.

References

- [1] Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*.
- [2] Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. *KDD*.
- [3] Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic Attribution for Deep Networks. *ICML*.
- [4] Slack, D., Hilgard, S., Jia, E., Singh, S., and Lakkaraju, H. (2020). Fooling LIME and SHAP. *AIES*.
- [5] Ghorbani, A., Abid, A., and Zou, J. (2019). Interpretation of Neural Networks is Fragile. *AAAI*.
- [6] Dombrowski, A.-K. et al. (2019). Explanations Can Be Manipulated and Geometry is to Blame. *NeurIPS*.
- [7] Madry, A. et al. (2018). Towards Deep Learning Models Resistant to Adversarial Attacks. *ICLR*.
- [8] Chen, J. et al. (2019). Robust Attribution Regularization. *NeurIPS*.
- [9] Boopathy, A. et al. (2020). Proper Network Interpretability Helps Adversarial Robustness. *ICML Workshop*.
- [10] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [11] Alvarez-Melis, D. and Jaakkola, T. S. (2018). On the Robustness of Interpretability Methods. *ICML Workshop*.